

Jour 2 — Après-midi : Chapitre 4 — Création de serveur MCP et agents autonomes

Niveau : DÉBUTANT — Durée : ~3h30 (après-midi du jour 2)

Récap du matin

Vous savez :

- Pourquoi un LLM seul ne suffit pas en entreprise.
- Comment marche un RAG (mini-RAG codé ce matin).
- Comment connecter votre IDE à un serveur MCP du marketplace.

Cet après-midi : **vous allez créer votre propre serveur MCP et comprendre les agents autonomes.**

Objectifs

1. Comprendre l'architecture d'un **serveur MCP custom**.
2. Implémenter un serveur MCP qui expose une **base SQLite** (3 outils).
3. Comprendre la **boucle d'un agent autonome** (ReAct).
4. Connaître les **patterns multi-agents**.
5. Identifier les **risques** : hallucinations, boucles infinies, sécurité.

Plan de l'après-midi

| Heure | Section | Durée | |---|---|---| | 14:00 | 1. Glossaire & vue d'ensemble | 15 min | | 14:15 | 2. Architecture d'un serveur MCP | 25 min | | 14:40 | 3. **Coder** un serveur MCP SQLite étape par étape | 50 min | | 15:30 | **Pause** | 15 min | | 15:45 | 4. Branchement client | 15 min | | 16:00 | 5. La boucle d'un agent ReAct | 25 min | | 16:25 | 6. Patterns multi-agents | 15 min | | 16:40 | 7. Risques et garde-fous | 15 min | | 16:55 | **TP 4** — Serveur MCP DB + démo agent | 1h05 | | 18:00 | Fin de la formation |

1. Glossaire

Mot	Définition simple
Serveur MCP	Un processus qui expose des outils selon le protocole MCP.

Mot	Définition simple
FastMCP	Une lib Python qui simplifie la création d'un serveur MCP.
stdio	Mode de transport : le client lance le serveur comme sous-processus.
Agent	Un LLM dans une boucle, équipé d'outils, qui poursuit un objectif.
ReAct	Reasoning + Acting : alterner pensée et action.
Tool call	Quand le LLM décide d'appeler un outil avec des arguments.
Boucle infinie	Quand l'agent ré-essaie indéfiniment la même action ratée.
Hallucination de table	Quand l'agent invente une colonne ou une table qui n'existe pas.
Read-only	Mode lecture seule (pas d'INSERT, UPDATE, DELETE...).

2. Architecture d'un serveur MCP custom

Cycle de vie en 5 étapes

1. **Démarrage** : le client lance le serveur comme sous-processus (`python serveur.py`).
2. **Handshake** : le client demande au serveur « *quelles capacités as-tu ?* ».
3. **Découverte** : le serveur annonce ses tools, ressources, prompts.
4. **Boucle** : le LLM décide d'appeler un tool → JSON-RPC → réponse → continue.
5. **Arrêt** : le client envoie un signal d'arrêt à la fin de la session.

Choix techniques

Choix	Recommandation débutant
Langage	Python (FastMCP, le plus simple)
Transport	stdio (local, pas de réseau, pas de sécurité à gérer)
Forme	1 script Python par serveur
Dépendances	<code>pip install mcp</code> (officiel Anthropic)

Anatomie d'un serveur FastMCP minimal

```
In [ ]: # =====
# Le serveur MCP le plus simple possible – "Hello World"
# =====

# --- Etape 0 : installer
# pip install mcp

# --- Etape 1 : importer la lib
from mcp.server.fastmcp import FastMCP
```

```
# --- Etape 2 : creer une instance de serveur (nom libre, c'est l'identifiant)
mcp = FastMCP("hello-world")

# --- Etape 3 : declarer un tool avec le decorateur @mcp.tool()
@mcp.tool()
def saluer(nom: str) -> str:
    """Renvoie une salutation polie.

    Args:
        nom: Le nom de la personne a saluer.

    Returns:
        Une chaine de salutation.
    """
    return f"Bonjour, {nom} !"

# --- Etape 4 : declarer un autre tool
@mcp.tool()
def additionner(a: int, b: int) -> int:
    """Additionne deux entiers."""
    return a + b

# --- Etape 5 : lancer Le serveur (en mode stdio par default)
if __name__ == "__main__":
    mcp.run() # ce processus va ecouter sur stdin/stdout
```

Ce qu'il faut retenir

- Le décorateur `@mcp.tool()` transforme une fonction Python en outil MCP.
- Les type hints sont importants : FastMCP les utilise pour générer le schéma JSON que le LLM voit.
- La docstring devient la description du tool que le LLM lit pour décider quand l'appeler.

💡 Plus votre docstring est claire, mieux l'agent choisira votre tool. Soignez les docstrings comme si vous écriviez de la documentation API.

3. Coder un serveur MCP SQLite — étape par étape

On va construire un serveur qui expose une base SQLite **en lecture seule**.

Pourquoi en lecture seule ?

Parce qu'un LLM **peut faire n'importe quoi**, y compris :

- générer un `DROP TABLE users`,
- exécuter un `DELETE FROM accounts` sans WHERE,
- consommer toute la mémoire avec une jointure cartésienne.

Règle d'or : un agent qui touche à une DB doit être **en lecture seule** par défaut. Les opérations destructives doivent passer par une validation humaine.

Les outils à exposer

Tool	Description	Sécurité
list_tables()	Liste les tables disponibles	Pas de risque
describe_table(table)	Schéma (colonnes, types)	Validation du nom de table
count_rows(table)	Nombre de lignes	Validation du nom de table
query(sql)	Exécute un SELECT	Regex anti-modification + LIMIT 1000

```
In [ ]: # =====
# Serveur MCP SQLite read-only – version commentée pour apprendre
# Fichier : serveur_sqlite.py
# =====

# --- Imports
import sqlite3                                # base de donnees locale
import re                                      # expressions regulieres pour la
import json                                    # pour l'audit log
import datetime                                # pour horodater l'audit
from pathlib import Path                       # gestion de chemins
from mcp.server.fastmcp import FastMCP        # framework MCP

# --- Configuration
mcp = FastMCP("sqlite-readonly")              # nom du serveur
DB_PATH = "exemple.db"                       # chemin de la base
AUDIT_LOG = Path("audit.jsonl")              # fichier d'audit

# --- Regex qui bloque tout mot-clef destructif
# \b = limite de mot. re.IGNORECASE = insensible a la casse.
MOTS_INTERDITS = re.compile(
    r"\b(insert|update|delete|drop|alter|create|truncate|attach|pragma)\b",
    re.IGNORECASE
)

# --- Fonction utilitaire : ouvrir la base avec un timeout de 5s
def ouvrir_connexion():
    """Retourne une connexion SQLite avec timeout 5s (anti-deadlock)."""
    return sqlite3.connect(DB_PATH, timeout=5)

# --- Fonction utilitaire : audit log
def enregistrer_audit(nom_tool, arguments, apercu_resultat):
    """Ecrit une ligne JSON par appel dans audit.jsonl."""
    ligne = {
        "horodatage": datetime.datetime.utcnow().isoformat(),
        "tool": nom_tool,
        "args": arguments,
        "aperçu": str(apercu_resultat)[:200] # on limite a 200 char
    }
```

```

with AUDIT_LOG.open("a", encoding="utf-8") as f:
    f.write(json.dumps(ligne) + "\n")

# --- Tool 1 : lister les tables
@mcp.tool()
def list_tables() -> list[str]:
    """Liste toutes les tables de la base."""
    with ouvrir_connexion() as con:
        # sqlite_master est la table systeme qui liste toutes les tables
        lignes = con.execute(
            "SELECT name FROM sqlite_master WHERE type='table'"
        ).fetchall()
    resultat = [r[0] for r in lignes] # on prend juste le nom
    enregistrer_audit("list_tables", {}, resultat)
    return resultat

# --- Tool 2 : decrir une table
@mcp.tool()
def describe_table(table: str) -> list[dict]:
    """Retourne le schema d'une table (colonnes, types, nullable)."""
    # Securite : valider que le nom ne contient que des caracteres safe
    if not re.match(r"^[a-zA-Z_][\w]*$", table):
        raise ValueError(f"Nom de table invalide : {table}")
    with ouvrir_connexion() as con:
        lignes = con.execute(f"PRAGMA table_info({table})").fetchall()
    resultat = [
        {"col": r[1], "type": r[2], "nullable": not bool(r[3])}
        for r in lignes
    ]
    enregistrer_audit("describe_table", {"table": table}, resultat)
    return resultat

# --- Tool 3 : compter les lignes
@mcp.tool()
def count_rows(table: str) -> int:
    """Retourne le nombre de lignes d'une table."""
    if not re.match(r"^[a-zA-Z_][\w]*$", table):
        raise ValueError(f"Nom de table invalide : {table}")
    with ouvrir_connexion() as con:
        (n,) = con.execute(f"SELECT COUNT(*) FROM {table}").fetchone()
    enregistrer_audit("count_rows", {"table": table}, n)
    return n

# --- Tool 4 : executer un SELECT (le plus risque)
@mcp.tool()
def query(sql: str) -> list[dict]:
    """Execute une requete SQL (lecture seule, max 1000 lignes, 5s)."""
    # Securite 1 : interdire les mots-cles destructifs
    if MOTS_INTERDITS.search(sql):
        raise ValueError("Seul SELECT est autorise. INSERT/UPDATE/DELETE/DROP refus")

    # Securite 2 : forcer un LIMIT si absent
    if "limit" not in sql.lower():
        sql = sql.rstrip(";") + " LIMIT 1000"

    # Execution

```

```
with ouvrir_connexion() as con:
    cur = con.execute(sql)
    colonnes = [d[0] for d in cur.description] # noms des colonnes
    lignes = [dict(zip(colonnes, r)) for r in cur.fetchall()]

    enregistrer_audit("query", {"sql": sql}, f"{len(lignes)} lignes")
    return lignes

# --- Lancement du serveur
if __name__ == "__main__":
    mcp.run()
```

Décortiquons les 6 garde-fous

#	Garde-fou	Code	Pourquoi
1	Mots-clés interdits	regex MOTS_INTERDITS	Bloque INSERT, UPDATE, DELETE, DROP, etc.
2	Validation nom de table	regex ^[a-zA-Z_][\w]*\$	Empêche les injections via nom de table
3	LIMIT automatique	si absent → ajouter LIMIT 1000	Évite de tout dumper
4	Timeout de connexion	timeout=5	Évite les deadlocks
5	Audit log	audit.jsonl	Traçabilité de tous les appels
6	Pas de mot de passe	utilisateur sans privilèges	Défense en profondeur

⚠ **À retenir** : un serveur MCP sans garde-fou = un compte admin donné à un stagiaire le premier jour. À éviter.

4. Branchement client (côté Claude Desktop)

Pour utiliser votre serveur, on l'ajoute à `claude_desktop_config.json` :

```
In [ ]: # =====
# Configuration cote client Claude Desktop
# =====
# Fichier : claude_desktop_config.json

import json

config = {
    "mcpServers": {
        # Nom Libre, on l'appelle "ma-db"
        "ma-db": {
            "command": "python", # interpret
            "args": [
                "/chemin/absolu/vers/serveur_sqlite.py" # IMPORTANT
```

```

    }
}

# Sauvegarder ce JSON dans le bon fichier, puis redémarrer Claude Desktop
print(json.dumps(config, indent=2))

# Verification :
# - Claude Desktop affiche un icône outils en bas à gauche.
# - Cliquer dessus : "ma-db" doit apparaître avec 4 tools.

```

5. La boucle d'un agent autonome — pattern ReAct

Définition

Un **agent** = un LLM **dans une boucle**, équipé d'outils, avec un objectif.

```

while not objectif_atteint:
    perception = observer_environnement()      # lire (via tools)
    reflexion  = llm.penser(perception)        # raisonner
    action     = llm.choisir_outil(reflexion)   # décider quoi faire
    resultat   = executer(action)              # appeler l'outil
    etat       = mettre_a_jour(etat, resultat)

```

Conditions d'arrêt — TRÈS IMPORTANT

Sans condition d'arrêt, votre agent peut **tourner à l'infini** et coûter cher. Donc, **toujours** définir :

- ✓ **Succès** : critère explicite (test passe, réponse trouvée).
- ✓ **Max d'étapes** : `max_steps=25` typiquement.
- ✓ **Budget tokens** : « plus de 50 000 tokens → arrêt ».
- ✓ **Erreur fatale** : exception non rattrapée → arrêt.

Pattern ReAct (Yao et al., 2022)

Le LLM alterne **explicitement** `Thought:` et `Action:` à chaque tour. La trace devient **auditable** :

```

Thought: Je dois trouver les utilisateurs avec une commande >
100€.
        Je vais d'abord vérifier qu'il y a bien une table
'orders'.
Action:  list_tables()
Result:  ['users', 'orders']

Thought: Parfait. Maintenant je regarde le schéma de 'orders'.
Action:  describe_table('orders')

```

Result: [{"col": "id"}, {"col": "user_id"}, {"col": "amount"}, ...]

Thought: Il y a une colonne amount. Je peux requeter.

Action: query("SELECT COUNT(DISTINCT user_id) FROM orders WHERE amount > 100")

Result: [{"COUNT(DISTINCT user_id)": 23}]

Thought: J'ai la reponse.

Action: FINAL_ANSWER("23 utilisateurs ont passe une commande > 100€")

C'est ce que font Claude Code, Devin, Cursor Agent, etc.

```
In [ ]: # =====
# Boucle ReAct minimaliste, illustrative (sans vrai LLM)
# =====

# --- On simule des outils (en vrai : Les outils du serveur MCP)
OUTILS = {
    "list_tables": lambda: ["users", "orders"],
    "describe_table": lambda table: [{"col": "id"}, {"col": "user_id"}, {"col": "am"},
    "query": lambda sql: [{"reponse": "23 utilisateurs"}],
}

# --- On simule le LLM : ici scenario en dur
def llm_decide(historique):
    """Le LLM regarde l'historique et decide de la prochaine action."""
    nb_etapes = len([h for h in historique if h["type"] == "action"])
    if nb_etapes == 0:
        return {"thought": "Je verifie les tables disponibles.",
                "action": ("list_tables", {})}
    elif nb_etapes == 1:
        return {"thought": "Je regarde le schema de orders.",
                "action": ("describe_table", {"table": "orders"})}
    elif nb_etapes == 2:
        return {"thought": "Je requete.",
                "action": ("query", {"sql": "SELECT COUNT(*) FROM orders WHERE amou"}
    else:
        return {"thought": "Termine.", "action": None}

# --- Boucle ReAct avec garde-fous
def agent(question, max_etapes=10):
    historique = [{"type": "question", "contenu": question}]
    for etape in range(max_etapes):
        decision = llm_decide(historique)
        print(f"[Etape {etape}] Thought: {decision['thought']}")
        if decision["action"] is None:
            print("[Fin] Objectif atteint.")
            return historique
        nom_outil, args = decision["action"]
        # Verification de securite : L'outil existe ?
        if nom_outil not in OUTILS:
            print(f"[Erreur] Outil inconnu : {nom_outil}")
            break
```



```

# Execution
resultat = OUTILS[nom_outil](**args)
print(f"[Etape {etape}] Action: {nom_outil}({args}) -> {resultat}")
historique.append({"type": "action", "outil": nom_outil, "args": args, "res
print("[Fin] Max etapes atteint.")
return historique

# --- Test
agent("Combien d'utilisateurs ont passe une commande > 100 € ?")

```

6. Patterns multi-agents

Quand un seul agent ne suffit pas, on compose plusieurs agents.

Pattern	Description	Quand l'utiliser
Pipeline	A → B → C en séquence	Tâches qui s'enchaînent (rechercher → résumer → publier)
Router	Un agent choisit lequel des N agents spécialisés activer	Support client (tri par intent)
Worker / Critic	Worker produit, critic relit et renvoie	Tâches qualité (review, rédaction)
Manager hiérarchique	Un manager pilote N sous-agents	Projet complexe (Devin, AutoGPT)
Debate / Self-consistency	N agents répondent en parallèle, on agrège	Décisions sensibles

Compromis

⚖️ Plus d'agents = **plus de fiabilité**, mais **explosion des coûts** et **latence**.

Règle pratique : commencez avec **1 agent**, ajoutez **1 critic** si besoin, escaladez seulement si nécessaire.

Coûts cachés

- 3 agents en parallèle = **3× le coût** pour le même temps.
- Un manager + 5 workers = la latence du plus lent + le coût total.
- Boucles infinies non détectées = facture explosive.

7. Risques et garde-fous

Risque 1 — Hallucinations

Le LLM invente un nom de colonne, une fonction, une lib.

Mitigation :

- Valider chaque action contre un schéma réel **avant** exécution.
- Forcer l'agent à appeler `describe_table` **avant** `query` .

Risque 2 — Boucles infinies

L'agent ré-essaie indéfiniment.

Mitigation :

- `max_steps` (typiquement 25).
- Hash des 3 derniers tours → si répétition, arrêt.
- Budget tokens absolu.

Risque 3 — Sécurité

Un LLM qui écrit du SQL = un nouveau vecteur d'injection.

Mitigation :

- Utilisateur DB **read-only**.
- Parser SQL avant exécution.
- Whitelist de schémas.
- Audit log immuable.

Risque 4 — Coûts incontrôlés

Un agent peut consommer des millions de tokens.

Mitigation :

- Plafond de coût par session.
- Alertes en temps réel.
- Kill-switch automatique.

Risque 5 — Confidentialité

Envoyer du code propriétaire à une API externe.

Mitigation :

- SLM local pour les tâches sensibles.
- Anonymisation préalable.
- DPA contractuel avec le fournisseur.

TP 4 — Serveur MCP DB + démo agent autonome

Durée : 1h05 — Modalité : binôme

Étape 1 — Préparer la base (10 min)

Exécuter le script `ressources/seed_db.py` qui crée `exemple.db` avec :

- 50 utilisateurs (id, email, name, role)
- 50 commandes (id, user_id, amount, status)

Étape 2 — Coder le serveur (30 min)

Reprendre le code de la section 3. Vous devez ajouter :

- Un 5^e outil : `top_clients(limit: int)` qui retourne les N clients avec le plus de commandes.
- Vérifier que `audit.jsonl` se remplit bien.

Étape 3 — Brancher le client (5 min)

Configurer Claude Desktop ou Continue avec votre serveur. Vérifier que **5 outils** apparaissent.

Étape 4 — Démonstrations guidées (20 min)

Dans une session :

1. « *Combien d'utilisateurs ont une commande de plus de 100 € ?* » → observer la trace.
2. « *Supprime la table orders.* » → vérifier que c'est bloqué.
3. « *Donne-moi les téléphones des utilisateurs.* » → observer la réaction (colonne inexistante).
4. Examiner `audit.jsonl` à la fin.

Livrables

- `serveur_sqlite.py` complet avec les 5 outils.
- `audit.jsonl` (traces).
- `tp4_rapport.md` : 1 paragraphe par démo (3-5 lignes chacun).

Corrigés : `corriges/jour2/CORRIGE_chapitre4.ipynb` .

Synthèse — Fin de la formation IA pour développeur

Ce que vous savez maintenant

- **Jour 1** : LLM, tokens, prompting structuré (ROCDC-IO).
- **Jour 2** : RAG, MCP, serveur custom, agents ReAct, garde-fous.

Les 7 réflexes à conserver

1. **Compter les tokens** avant de payer.
2. **Préférer un SLM local** pour les tâches simples.
3. **Structurer ses prompts** avec ROCDC-IO.
4. **Forcer l'idempotence** en CI/CD (temperature=0).
5. **Garder l'humain dans la boucle** pour les actions destructives.
6. **Audit log** sur tous les serveurs MCP.
7. **Max_steps + budget** sur tous les agents.

Pour aller plus loin

- Spec MCP : <https://modelcontextprotocol.io>
- Anthropic — *Building effective agents* (décembre 2024)
- ReAct : Yao et al., arXiv:2210.03629
- LangGraph, CrewAI, AutoGen — frameworks d'orchestration
- SWE-bench, AgentBench — benchmarks pour évaluer un agent

Évaluation finale (15 min)

- QCM 10 questions : `exercices/qcm_final.md`
- Auto-évaluation : noter chacun des 6 objectifs de la formation sur 5.

 **Bravo !** Vous êtes prêt à expérimenter chez vous, et à pousser progressivement vos premiers usages en production.